Internal Covariate Shift refers to the **change in the distribution of inputs to a layer during the training of a neural network** as the parameters of previous layers keep changing.

As earlier layers learn and their weights change, the output they produce for the next layer also keeps changing. This makes the next layer continuously adapt to new input distributions.

Example

Consider a neural network:

Input → Layer 1 → Layer 2 → Layer 3 → Output

During training:

1. Layer 1 updates its weights.
2. Its output changes.
3. Therefore, the input received by Layer 2 changes.
4. Layer 2 has to keep adjusting to these changing inputs.
5. This can make training slower or more difficult.

This changing distribution is called **Internal Covariate Shift**.

Batch normalization can help to solve **Internal Covariate Shift**.

Write the steps in batch normalization

Eg: **Internal Covariate Shift**.

Suppose Layer 1 initially produces:

[2,4,6,8]

These values become the input to Layer 2.

After some training, the weights of Layer 1 change. Now it might produce:

[20,40,60,80]

So Layer 2 is suddenly receiving inputs with a very different distribution.

Later, Layer 1 changes again:

[0.2,0.4,0.6,0.8]

Again, Layer 2 receives a different distribution.

So Layer 2 has to continuously adapt to changing inputs.

Why is this a problem?

Imagine you are training Layer 2.

Initially, it learns:

“When I receive values around 2–8, I should produce this output.”

But while it is learning, Layer 1 changes its weights.

Now Layer 2 receives values around:

20-80

Layer 2 has to adjust again.

Then the distribution changes again.

Therefore:

Changing input distribution → continuous adaptation → potentially slower or less stable training.

We can apply BN